

CHAPTER 3

SEQUENCES: STRINGS, TUPLES, AND LISTS

3.1 *Strings*3.2 *The Type Bytes*3.3 *Tuples*3.4 *Lists*3.5 *Set Types*3.6 *Summary***In this chapter**

It was mentioned in [Chapter 2](#) that **for** loops in Python are different from those found in many other languages. In Java and C++, a **for** loop has a very explicit increment; a **for** statement looks like this in Java:

From this it can be inferred that the variable `i` will start out as 0, and so long as `i` is less than 10 the loop will continue. After each iteration the value of `i` will be increased by 1, and then the condition is tested again.

In Python the iteration is more implicit, with the loop control variable taking on one of a set of values in turn. There is an implication here, too, that there is a kind of thing, a type that a variable can have, that amounts to a list or sequence of other, simpler things. This is true, and using variables having these types is an essential part of writing useful and effective code. Python offers *strings*, *tuples*, and *lists* as objects that consist of multiple parts. They are called *sequence* types. An integer or a float is a single number, whereas a sequence type consists of a collection of items, each of which is a number or a character. Each member of a sequence is given a number based on its position: the first element in the sequence is given 0, the second is 1, and so on. This is a fundamental data structure in Python and has influenced the syntax of the language.

Strings are familiar objects and have been used in programs already, so the discussion will begin there.

3.1 Strings

A *string* is a sequence of characters. The word *sequence* implies that the order of the characters within the string matters, and that is certainly true. Strings most often represent the way that communication between a computer and a human takes place. Human language consists of words and phrases, and each word or phrase would be a string within a program. The order of the characters within a word matters a great deal to a

human because some sequences are words and others are not. The string "last" is a word, but "astl" is not. Also, the strings "salt" and "slat" are words and use exactly the same characters as "last" but in a different order.

Because order matters, the representation of a string on a computer will impose an order on the characters within, and so there will be a first character, a second, and so on, and it should be possible to access each character individually. A string will also have a *length*, which is the number of characters within it.

A computer language will provide specific things that can be done to something that is a string: these are called operations, and a type is defined at least partly by what operations can be done to something of that type. Because a string represents text in the human sense, the operations on strings should represent the kinds of things that would be done to text. This would include printing and reading, accessing any character, linking strings into longer strings, searching a string for a particular word, and so on.

The examples of code written so far use only string *constants*. These are simply characters enclosed in either single or double quotes. Assigning a string constant to a variable causes that variable to have the string type and gives it a value. So the statements:

```
name = "John Doe"
```

```
address = '121 Second Street'
```

cause the variables named **name** and **address** to be strings with the assigned value. Note that either type of quote can be used, but a string that begins with a double quote must end with one.

A string behaves as if its characters are stored as consecutive characters in memory. The first character in a string is at location or index 0, and can be accessed using square brackets after the string name. Using the definitions above, `name[0]` is "J" and `name[5]` = "D." If an index is specified that is too large, it results in an error, because it amounts to an attempt to look past the end of the string.

How many characters are there in the string **name**? The built-in function **len()** will return the length of the string. The largest legal index is one less than this value: the first character of a string **name** has index 0, and the final one has index 7; the length is 8. Thus, any index between 0 and **len(name)-1** is legal. The following code prints all of the characters of **name** and can be thought of as the basic pattern for code that scans through the characters in strings:

```
for i in range(0, len(name)):
    print (name[i], end="")
```

This may be a little confusing, but remember that the **range(0,n)** does not include **n**. This loop runs through values of **i** from 0 to **len(name)-1**.

Some languages have a *character* type, but Python does not. A string of length one is what Python uses instead. A component of a string is therefore another string. The first character of the string `name`, which is `name[0]`, is "J," the string containing only one character.

3.1.1 Comparing Strings

Two strings can be compared in the same manner as are two integers or real numbers, by using one of the relational operators `==`, `!=`, `<`, `>`, `<=` or `>=`. What it means for two strings to be equal is simple and reasonable: if each corresponding character in two strings is the same, then the strings are equal. That is, for strings `a` and `b`, if `a[0] == b[0]`, and `a[1] == b[1]` and so on to the final character `n`, and `a[n] == b[n]`, then the two strings `a` and `b` are equal and `a == b`. Otherwise, `a != b`. By the way, this implies that equal strings have the same length.

What about inequalities? Strings in real life are often sorted in alphabetical order. Names in a telephone book, files in a doctor's office, and books in a store: these tend to appear in a logical order based on the alphabet. This is also true in Python. The string "abc" is less than the string "def," for example. Why? Because the first letter in "abc" comes before the first letter in "def"; in other words, `"abc"[0] < "def"[0]`. Yes, characters in string constants can be accessed using their index.

A string `s1` is less than string `s2` if all characters from 0 through `k` in the two strings are equal, and `s1[k+1] < s2[k+1]`. So the following statements are true:

```
"abcd" < "abce"
"123" < "345"
"ab " < "abc"
```

In the last example, the space character "." is smaller than (i.e., comes before) the letter "c." What if the strings are not the same length? The string `"ab" < "abc"`, so if two strings are equal to the end of one of them, then the shorter one is considered to be smaller. These rules are consistent so far with those taught in grade school for alphabetization. Trailing spaces do not matter. *Leading* spaces can matter, because a space comes before any alphabetic character; that is, `" " < "a"`. Thus `"ab" > "z"`.

Digits come before lowercase letters. `"1" < "a"`, and `"1a" < "a1"`. **Most importantly, uppercase comes before lowercase, so `"John" < "john"`.** All of these rules are consistent with those that secretaries understand when filing paper documents. As an example that compares strings, consider the following:

```
a = "J"
b = "j"
c = "1"
if b < c:
```

```

    print ("Lcase < numbers")
else:
    print("Lcase > numbers")
if a<c:
    print ("Ucase < numbers")
else:
    print("Ucase > numbers")

```

This results in the output:

```
Lcase > numbers
```

```
Ucase > numbers
```

Problem: Does a City Name, Entered at the Console, Come before or after the Name Denver?

This involves reading a string and comparing it against the constant string "Denver." Let the input string be read into a variable named **city**. Then the answer is:

```

city = input()
if city < "Denver":
    print ("The name given comes before Denver in an
           alphabetic list")
elif city > "Denver":
    print ("The name given comes after Denver in an
           alphabetic list")
else:
    print ("The name given was Denver")

```

If "Chicago" is typed at the console as input, the result is:

```
Chicago
```

```
The name given comes before Denver in an alphabetic list
```

However, if case is ignored and "chicago" is typed instead, then the result is:

```
chicago
```

```
The name given comes after Denver in an alphabetic list
```

because, of course, the lowercase "c" comes (as do *all* lowercase letters) after the uppercase "D" at the beginning of "Denver."

3.1.2 Slicing – Extracting Parts of Strings

To a person, a string usually contains words and phrases, which are smaller parts of a string. Identifying individual words is important. To Python this is true also. A Python program consists of statements that contain individual words and character sequences that each have a particular meaning. The words "if," "while," and "for" are

good examples. Individual characters can be referenced through indexing, but can words or collections of characters be accessed? Yes, if the location (index) of the word is known.

Problem: Identify a "Print" Statement in a String

The statement:

```
print ("Lcase < numbers")
```

appears in the example program above. This can be thought of as a string, and assigned to a variable:

```
statement = 'print ("Lcase < numbers")'
```

Question: is this a **print** statement? It is if the first five characters are the word "print." Each of those characters could be tested individually using:

```
if statement[0] == 'p':
    if statement[1] == 'r':
        if statement[2] == 'i':
            if statement[3] == 'n':
                if statement[4] == 't':
                    if statement[5] == ' ':
                        # This is a print statement.
```

This is pretty ugly, and is something that is needed often enough that Python offers a nicer way to do it. A *slice* is a set of continuous characters within a string. This means their indices are consecutive, and they can be accessed as a sequence by specifying the range of indices within brackets. The situation above concerning the print statement could be done like this:

```
if statement[0:5] == "print":
```

The slice here does not include character 5, but is 5 characters long including characters 0 through 4 inclusive. A slice from **i** to **j** (i.e., **x[i:j]**) does not include character **j**. This means that the following statements produce the same result:

```
fname[0]
```

```
fname[0:1]
```

If the first index is omitted, then the start index is assumed, so the statement:

```
if statement[0:5] == "print":
```

is the same as:

```
if statement[:5] == "print":
```

If the second index is omitted, then the last legal index is assumed, which is to say the index of the final character. So the assignment:

```
str = statement[6:]
```

results in the value of `str` being `"("Lcase < numbers")"`. Both indices can be omitted, which does sound silly, but really just means from the first to the last character, or the entire string.

3.1.3 Editing Strings

Python does not allow the modification of individual parts of a string. That is, things like:

```
str[3] = "#"
str[2:3] = ".."
```

are not allowed. So how can strings be modified? For example, consider the string variable:

```
fname = "image"
```

If this is supposed to be the name of a JPEG image file, then it must end with the suffix `".jpg"`.

Problem: Create a JPEG File Name from a Basic String

The string `fname` can be edited to end with `".jpg"` in a few ways, but the easiest one to use is the concatenation operator `+`.

To *concatenate* means "to link or join together." If the variables `a` and `b` are strings, then `a+b` is the string consisting of all characters in `a` followed by all characters in `b`; the operator `+` in this context means to concatenate, rather than to add numerically. The designers of Python and many other languages that implement this operator think of concatenation as string addition.

To use this to create the image file name, simply concatenate `".jpg"` to the string `fname`:

```
fname = fname + ".jpg"
```

The result is that `fname` contains `"image.jpg"`.

File suffixes are very often the subject of string manipulations and provide a good example of string editing. For instance, given a file name stored as a string variable `fname`, is the suffix `".jpg"`? Based on the preceding discussion, the question can be answered using a simple `if` statement:

```
if fname[len(fname)-4:len(fname)] == '.jpg':
```

Using a slice it could also take the form:

```
if fname[len(fname)-4:] == ".jpg"
```

A valuable thing to know is that negative indices index from the *right*-hand side of the string; that is, from the end. So `fname[-1]` is the final character in the string, `fname[-2]` is the one previous to that, and so on. The last 4 characters, the suffix, would be captured by using `filename[-4:]`.

Problem: Change the Suffix of a File Name

Some individuals use the suffix `".jpeg"` instead of `".jpg."` Some programs allow this, others do not. Some code that would detect and change this suffix would be:

```
if fname[len(fname)-5:] == ".jpeg": # identify the jpeg

    # suffix

    fname = fname[0:len(fname)-5] # remove the last five

    # characters

    fname = fname + ".jpg"      # append the correct

    # suffix
```

Problem: Reverse the Order of Characters in a String

There are things about any programming language that could be considered to be "idioms." These are things that a programmer experienced in the use of that language would consider normal use, but that others might consider odd. This problem exposes a Python idiom. Given what is known so far about Python, the logical approach to string reversal might be as follows:

```
# city has a legal value at this point

k = len(city)

for i in range(0,len(city)):

    city = city + city[k-i-1]

city = city[len(city)//2:]
```

This reverses the string named `city` that exists prior to the loop and creates the reversed string. It does so in the following way:

1. Let `i` be an index into the string `city`, starting at 0 and running to the final character.
2. Index a character from the end of the string, starting at the final character and stepping backwards to 0. Since the last character is `len(city)` and the current index is `i`, the character to be used in the current iteration would be `k-i-1` where `k` is the length of the original string.
3. Append `city[k-i-1]` to then end of the string. Alternatively, a new string `rs` could be created and this character appended to it during each iteration.
4. After all characters have been examined, the string `city` contains the original string at the beginning and the reversed string at the end. The first characters can be removed, leaving the reversed string only.

An experienced Python programmer would do this differently. The syntax for taking a slice has a variation that has not been discussed; a third parameter exists. A string slice can be expressed as:

```
myString[a:b:c]
```

where **a** is the starting index, **b** is the final index+1, and **c** is the increment. Increment? If:

```
str = "This string has 30 characters."
```

Then `str[0:30:2]` is "Ti tighs3 hrces," which is every second character. The increment represents the way the string is sampled, that is, every **increment** character is copied into the result. Most relevant to the current example, the increment can be negative. The idiom for reversing a string is:

```
print (str[::-1])
```

As has been explained, the value of `str[:]` is the whole string. Specifying an increment of -1 implies that the string is scanned from 0 to the end, but in reverse order. This is far from intuitive, but is probably the way that an experienced Python programmer would reverse a string. Any programmer should use the parts of any language that they comprehend very well, and should keep in mind the likely skill set of the people likely to read the code.

Problem: Is a Given File Name That of a Python Program?

A Python program terminates with the suffix ".py." An obvious solution to this problem is to simply look at the last 3 characters in the string **s** to see if they match that suffix:

```
if s[len(s)-3:len(s)] == '.py':
    print ("This is a Python program.")
```

Perhaps. But is "PROGRAM.PY" a legal Python program? It happens that it is. So is "program.Py" and "program.pY." What can be done here?

3.1.4 String Methods

A good way to do the test in this case is to convert the suffix to all upper- or all lowercase before doing the comparison. Comparing against ".py" means converted to lowercase, which is done by using a built-in *method* named **lower**:

```
s1 = s[len(s)-3:len(s)]
if s1.lower() == '.py':
    print ("This is a Python program.")
```

The variable **s1** is a string that will contain the final 3 characters of **s**. The expression **s1.lower()** creates a copy of **s1** in which all characters are lowercase. It's

called a *method* to distinguish it from a function, but they are very similar things. You should recall that a method is simply a function that belongs to one type or class of objects. In this case **lower()** belongs to the type (or class) *string*. There could be another method named **lower()** that belongs to another class and that did a completely different thing. The dot notation indicates that it is a method, and what class it belongs to: the same class of things that the variable belongs to. In addition, the variable itself is really the first parameter; if **lower** were a function, then it might be called by **lower(s1)** instead of **s1.lower()**. In the latter case, the “.” is preceded by the first parameter.

Strings all have many methods. In the table below the variable **s** is the *target* string, the one being operated upon. This means that the method names below will appear following “s.”, as in **s.lower()**. Let the value of **s** be given by **s = “hello to you all.”** These methods are intended to provide the operations needed to make the string type in Python function as a major communication device from humans to a program.

Method	Explanation (What is returned?)	Example
<code>capitalize()</code>	Returns the target string but with the first letter capitalized.	<code>s.capitalize() == "Hello to you all."</code>
<code>count(str,beg=0, end=len(s))</code>	Returns a count of how many times the string <code>str</code> occurs in the target. If values for <code>beg</code> and <code>end</code> are given, then the count is performed using only character indices between <code>beg</code> and <code>end</code> .	<code>s.count("ll") == 2</code>
<code>endswith(suffix, beg=0, end=len(s))</code>	Returns <code>True</code> if the target string ends with the given suffix and return <code>False</code> otherwise. If <code>beg</code> and <code>end</code> are given, then do the test on the substring between <code>beg</code> and <code>end</code> .	<code>s.endswith('ll.') == True</code>
<code>find(str, beg=0, end=len(string))</code>	If the string <code>str</code> appears with the target string, then return the index at which it occurs; return -1 if it does not occur. If <code>beg</code> and <code>end</code> are provided, then use the substring from <code>beg</code> to <code>end</code> .	<code>s.find("you") == 9</code>
<code>index(str, beg=0, end=len(string))</code>	<code>Index</code> is the same as <code>find</code> except that it will raise an exception if the string <code>str</code> does not occur in the target	<code>s.index("you") == 9</code>
<code>isdigit()</code>	Returns <code>True</code> if the target string contains only digits and <code>False</code> otherwise.	<code>s.isdigit() == False</code>
<code>islower()</code>	Returns <code>True</code> if the target string has at least 1 alphabetic character and all alphabetic characters are lowercase. Returns <code>False</code> otherwise.	<code>s.islower() == True</code>
<code>isspace()</code>	Returns <code>True</code> if the target string contains only whitespace characters and returns <code>False</code> otherwise.	<code>s.isspace() == False</code>

Method	Explanation (What is returned?)	Example
<code>isupper()</code>	Returns True if <code>s</code> has at least one alphabetic character and all alphabetic characters are uppercase. Returns False otherwise.	<code>s.isupper() == False</code>
<code>lower()</code>	Converts all uppercase letters in string to lowercase.	<code>s.lower() == s</code>
<code>replace(old, new [, max])</code>	Replaces all occurrences of the string <code>old</code> in the target with the string <code>new</code> . If <code>max</code> is specified, replace at most <code>max</code> instances.	<code>s.replace("you all", "y'all") == "hello to y'all."</code>
<code>split(str="", num=string.count(str))</code>	Returns a list of substrings obtained from the target using <code>str</code> as a delimiter. Space is the default for <code>str</code> . Subdivide at most <code>num</code> times if that is specified (see: Chapter 3, section 3).	<code>s.split(" ") == ["hello", "to", "you", "all"]</code>
<code>splitlines(num=string.count('\n'))</code>	Splits the target string at all (or <code>num</code> , if it is specified) NEWLINES and returns a list of each line with the NEWLINES removed.	<code>s.splitlines() == "hello to you all."</code>
<code>upper()</code>	Converts the lowercase letters in string to uppercase.	<code>s.upper() == "HELLO TO YOU ALL."</code>

3.1.5 Spanning Multiple Lines

Text as seen in human documents may contain many characters, even multiple lines and paragraphs. A special delimiter, the *triple quote*, is used when a string constant is to span many lines. This has been mentioned previously in the context of multiline comments. The regular string delimiters will terminate the string at the end of the line. The triple quote consists of either of the two existing delimiters repeated three times. For example, to assign the first stanza of Byron's poem "She Walks in Beauty" to the string variable `poem`:

```
poem = '''She walks in beauty like the night
Of cloudless climes and starry skies,
And all that's best of dark and bright
Meets in her aspect and her eyes;
Thus mellow'd to that tender light
Which Heaven to gaudy day denies.'''
```

When **poem** is printed the line endings appear where they were placed in the constant. This example is a particularly good one in that most poems require that lines end precisely where the poet intended.

Another example of a string that must be presented just as typed is a Python program. A program can be placed in a string variable using a triple quote:

```
program = """list = [1,2,4,7,12,15,21]
for i in list:
    print(i, i*2)"""
```

When printed this string has the correct form to be executed by Python. In fact, the following statement will actually execute the code in the string:

```
exec (program)
```

3.1.6 For Loops Again

Earlier in this section a for loop was written to print each character in the string. That loop was:

```
for i in range(0, len(name)):
    print (name[i], end="")
```

Obviously the string could have been printed using:

```
print(name)
```

but it was being used as an example of indexing individual components within the string. The characters do not need to be indexed explicitly in Python; the loop variable can be assigned the value of each component:

```
for i in name:
    print (i, end="")
```

In this case the value of **i** is the value of the component, not its index. Each component of the string is assigned to **i** in turn, and there is no need to test for the end of the string or to know its length. This is a better way to access components in a string and, as it happens, can be used with all sequence types. Whether an index is used or the components are pulled out one at a time depends on the problem being solved; sometimes the index is needed, other times it is not.

3.2 THE TYPE *BYTES*

A *string* is a sequence of characters, a sequence being defined as a collection within which order matters. *Strings* are commonly used for communication between computers and humans: to print headings and values on the screen, and to read objects in character string form. Humans deal with characters very well. The type *bytes* represents a sequence

of integers, albeit small ones. A *bytes* object of length 1 is an 8-bit integer, or a value between 0 and 255. A *bytes* object of length greater than 1 is a sequence of small integers. To be clear, if **s** is a *string* and **b** is a *bytes* then:

s[i] is a character

b[i] is a small integer

A string constant (literal) is a sequence of characters enclosed in quotes. A bytes literal is a sequence of characters enclosed in quotes and preceded by the letter "b." Thus:

'this is a string'

is a *string*, whereas:

b'this is a string'

has type *bytes*. Any method that applies to a *string* also applies to a *bytes* object, but bytes objects have some new ones. In particular, to convert a bytes object to a string, the **decode()** method is used and a character encoding should be given as the parameter. If no parameter is given, then the decoding method is the one currently being used. There are a few possible decoding methods (e.g., "utf-8"). So to convert a bytes object **b** to a character string **s**, the following would work:

s = b.decode ("utf-8")

A question remains: "why is the bytes type needed?" What is it used for? Because (and this is a little ahead of what is needed) it implements the *buffer interface*. Certain file operations require a buffer interface to accomplish their tasks. Anything read from some specific types of file will be of the type *bytes*, for example, as it has that interface. This will be discussed further in Chapters 5 and 8, but for the moment it simply serves to explain why the type exists at all. Other than the buffer interface, the bytes type is very much like a string, and can be converted back and forth.

3.3 TUPLES

A *tuple* is almost identical to a string in basic structure, except that it is composed of arbitrary components instead of characters. The quotes can't be used to delimit a tuple because a string can be a component, so a tuple is generally enclosed in parentheses. The following are tuples:

tup1 = (2, 3, 5, 7, 11, 13, 17, 19) # Prime numbers under 20

**tup2 = ("Hydrogen","Helium","Lithium","Beryllium","Boron",
"Carbon")**

tup3 = "hi", "ohio", "salut"

If there is only one element in a tuple, there should be a comma at the end:

```
tup4 = ("one",)
tup5 = "two",
```

That's because it would not be possible otherwise to tell the difference between a tuple and a string enclosed in parentheses. Is (1) a tuple? Or is it simply the number 1?

A tuple can be empty:

```
tup = ()
```

Because they are like strings, each element in a tuple has an index, and they begin at 0. Tuples can be indexed and sliced, just like strings. So:

```
tup1[2:4] is (5, 7)
```

Concatenation is like that of strings too:

```
tup4 = tup4 + tup5 # yields tup4 = ('one', 'two')
```

As is the case with strings, the index -1 gives the last value in the tuple, -2 gives the second last, and so on. So in the example above, `tup2[-1]` is "Carbon." Also, like strings, the tuple type is immutable; this means that elements in the tuple can't be altered. Thus, statements such as:

```
tup1[2] = 6
tup3[1:] "bonjour"
```

are not allowed and will generate an error.

Tuples are an intermediate form between strings, which have just been discussed, and lists, which will be discussed next. They are simpler to implement than *list* (are *lightweight*) and are more general than strings.

Are tuples useful? Yes, it turns out, and part of their use is that they underlie other aspects of Python.

3.3.1 Tuples in For Loops

Sequences can be used in a **for** loop to control the iteration and assign to the loop control variable. Tuples are interesting in this context because they can consist of strings, or integers or floats. The loop:

```
for i in ("Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"):
```

will iterate 6 times, and the variable `i` will take on the values in the tuple in the order specified. The variable `i` is a string in this case. In cases where the types in the tuple are mixed, things are more complicated.

Problem: Print the Number of Neutrons in an Atomic Nucleus

Consider the tuple:

```
atoms=("Hydrogen",1,"Helium",2,"Lithium",3,"Beryllium",4,  
      "Boron",5,"Carbon",6)
```

and the loop

```
for i in atoms:  
    print (i)
```

This prints:

```
Hydrogen  
1  
Helium  
2  
Lithium  
3  
Beryllium  
4  
Boron  
5  
Carbon  
6
```

The number following the name of the element is the atomic number of that element, the number of protons in the nucleus. In this case the type of the variable **i** alternates between string and integer. For elements with a low atomic number (less than 21), a good guess for the number of neutrons in the nucleus is twice the number of protons. The problem is that some of the components are strings and some are integers. The program should only do the calculation when it is in an iteration having an integer value for the loop variable, because a string can't be multiplied by two.

A built-in function that can be of assistance is **isinstance**. It takes a variable and a type name and returns **True** if the variable is of that type and **False** otherwise. Using this function, here is a first stab at a program that makes the neutron guess:

```
atoms=("Hydrogen",1,"Helium",2,"Lithium",3,"Beryllium",4,  
      "Boron",5,"Carbon",6)  
  
for i in atoms:  
    if isinstance(i, int):  
        j = i*2
```

```

    print ("has ", i, "protons and ", j, " neutrons.")
else:
    print ("Element ", i)

```

In other words, in iterations where **i** is an integer as determined by **isinstance**, then **i** can legally be multiplied by 2 and the guess about number of neutrons can be printed.

Another way to solve the same problem would be to index the elements of the tuple. Elements 0,2,4, etc. (even indices) refer to element names, while the others refer to atomic numbers. This code would look as follows:

```

atoms=("Hydrogen",1,"Helium",2,"Lithium",3,"Beryllium",4,
"Boron",5,"Carbon",6)
for i in range(0,len(atoms)):
    if i%2 == 1:
        j = atoms[i]*2
        print ("has ", atoms[i], "protons and ", j,
            " neutrons.")
    else:
        print ("Element ", atoms[i])

```

Note that in this case the loop variable is always an integer, and is not an element of the tuple but is an index at which to find an element. That's why the expression **atoms[i]** is used inside the loop instead of simply **i** as before.

3.3.2 Membership

Tuples are not sets in the mathematical sense, because an element can belong to a tuple more than once, and there is an order to the elements. However, some set operations could be implemented using tuples by looking at individual elements. Set union and intersection, for example. The *intersection* of two sets A and B is the set of elements that are members of A and also members of B. The membership operator for tuples is the key word **in**:

If 1 in tuple1:

The intersection of A and B, where A and B are tuples, could be found using the following code:

```

for i in A:
    if i in B:
        C = C + i

```

The tuple C will be the intersection of A and B. It works by taking each known element of A and testing to see if it is a member of B; if so, it is added to C.

Problem: What Even Numbers Less than or Equal to 100 are Also Perfect Squares?

This could be expressed as a set intersection problem. The set of even numbers less than 100 could be enumerated (this is not actual code):

```
A = (2,4,6,8,10 ... and so on
```

Or could be generated within a loop:

```
A = ()      # Start with an empty tuple
for i in range(0,51): # for appropriate integers
    A = A + (i*2,) # add the next even number to the
                  # tuple
# Can't simply use A+i because i is integer, not a tuple.
```

Similarly, the perfect squares could be enumerated:

```
B = (4,9,16,25,36,49,64,81,100)
```

Or, again, created in a loop:

```
B = ()
for i in range(0,11):
    B = B + ((i*i),)
```

Now the set A can be examined, element by element, to see which members also belong to B:

```
C = ()
for i in A:
    if i in B:
        C = C + (i,)
```

The result is: (0, 4, 16, 36, 64, 100).

Two important things are learned from this. First, when constructing a new tuple from components, one can begin with an empty tuple. Second, individual components can be added to a tuple using the concatenation operator "+," but the element should be made into a tuple with one component before doing the concatenation.

3.3.3 Delete

A tuple is *immutable*, meaning that it cannot be altered. Individual elements can be indexed but not changed or deleted. What can be done is to create a new tuple that has new elements; in particular, deleting an element means creating a new tuple that has all of the other elements except the one being deleted.

Problem: Delete the Element Lithium from the Tuple Atoms, along with Its Atomic Number.

Going back to the tuple `atoms`, deleting one of the components—in particular, *Lithium*—begins with determining which component *Lithium* is; that is, what is its index? So start at the first element of the tuple and look for the string "Lithium," stopping when it is found.

```
for i in range(0, len(atoms)):
    if atoms[i] == "Lithium": # Found it at location i
        break;
else:
    i = -1    # not found
```

Knowing the index of the element to be deleted, it is also known that all elements before that one belong to the new tuple and all elements after it do too. The elements before element `i` can be written as `atoms[0:i]`. Each element consists of a string and an integer, and assuming that both are to be deleted means that the elements following element `i` are `atoms[i+2:]`. In general to delete one element the second half would be `atoms[i+1:]`. Finishing the code that deletes "Lithium":

```
if i >= 0:
    atoms = atoms[0:i] + atoms[i+2:]
```

So the tuple `atoms` has not been altered so much as it has been replaced completely with a new tuple that has no *Lithium* component.

3.3.4 Update

Again, because a tuple is *immutable*, individual elements cannot be changed. A new tuple can be created that has new elements; in particular, updating an element means creating a new tuple that has all of the other elements except the one being updated, and that includes the new value in the correct position.

Problem: Change the Entry for *Lithium* to an Entry for *Oxygen*

An update is usually a deletion followed by the insertion or addition of a new component. A deletion was done in the previous section, so what remains is to add a new component where the old one was deleted. Inserting the element *Oxygen* in place of *Lithium* would begin in the same way as the simple deletion already implemented:

```
for i in range(0, len(atoms)):
    if atoms[i] == "Lithium": # Found it at location i
        break;
else:
    i = -1    # not found
```

Next, a new tuple for Oxygen is created:

```
newtuple = ("Oxygen", 8)
```

And finally this new tuple is placed at location `i` while *Lithium* is removed:

```
if i>=0:
    atoms = atoms[0:i] + newtuple + atoms[i+2:]
```

However, an update may not always involve a deletion. If *Lithium* is not a component of the tuple `atoms`, then perhaps *Oxygen* should be added to `atoms` anyway. Where? How about at the end?

```
else: # If i is -1 then the new tuple goes at the end
    atoms = atoms + newtuple
```

3.3.5 Tuple Assignment

One of the unique aspects of Python is so-called *tuple assignment*. When a tuple is assigned to a variable, the components are converted into an internal form that is the one tuples always use. This is called tuple packing, and is has already been encountered:

```
atoms=("Hydrogen",1,"Helium",2,"Lithium",3,"Beryllium",4,
      "Boron",5,"Carbon",6)
```

What is really interesting is that tuple *unpacking* can also be used. Consider the tuple:

```
srec = ('Parker', 'Jim', 1980, 'Math 550', 'C+', 'Cpsc 302', 'A+')
```

which is a tuple packing of a student record. It can be unpacked into individual variables in the following way:

```
(fname, lname, year, cmin, gmin, cmax, gmax) = srec
```

Which is the same as:

```
fname = srec[0]
lname = srec[1]
year = srec[2]
cmin = srec[4]
gmin = srec[5]
cmax = srec[6]
gmax = srec[7]
```

Of course, the implication is that `N` variables can be assigned the value of `N` expressions or variables “simultaneously” if both are written as tuples. Examples would be:

```
(a, b, c, d, e) = (1,2,3,4,5)
```

```
(f, g, h, i, j) = (a, b, c, d, e)
```

The expression

```
(f, g, h, i, j) = 2 ** (a,b,c,d,e)
```

is invalid because the left side of `***` is not a tuple, and Python won't convert 2 into a tuple. Also:

```
(f, g, h, i, j) = (2,2,2,2,2) ** (a,b,c,d,e)
```

is also invalid because `***` is not defined on tuples, nor are other arithmetic operations. As with strings, `+` means concatenation, though, so `(1,2,3) + (4,5,6)` yields `(1,2,3,4,5,6)`.

Exchanging values between two variables is a common thing to do. It's an essential part of a sorting program, for example. The exchange in many languages requires three statements: a temporary copy of one of the variables has to be made during the swap:

```
temp = a
```

```
a = b
```

```
b = temp
```

Because of the way that tuples are implemented, this can be performed in one tuple assignment:

```
(a,b) = (b,a)
```

This is a little obscure, not to an experienced Python programmer but certainly to a beginner, and often to experienced programmers in other languages. A Java programmer could see what was meant, but initially the rationale would not be obvious. This statement deserves a comment such as "perform an exchange of values using a tuple assignment."

3.3.6 Built-In Functions for Tuples

As examples for the table below, use the following:

```
T1 = (1,2,3,4,5)
```

```
T2 = (-1,2,4,5,7)
```

Function	Explanation (What Is Returned?)	Example
<code>len(T1)</code>	Gives the number of components that are members of T1.	<code>len(T1) == 5</code>
<code>max(T1)</code>	Returns the largest element that is a component of T1.	<code>max(T1) == 5</code> <code>max(T2) == 7</code>
<code>min(T1)</code>	Returns the smallest element that is a component of T1.	<code>min(T1) == 1</code> <code>min(T2) == -1</code>

In addition, tuples can be compared using the same operators as for integers and strings. Comparison is done on an element-by-element basis, just as it is with strings. In the example above, **T1>T2** because at the first location where the two tuples differ (the initial component), the element in **T1** is greater than the corresponding element in **T2**. It is necessary for the corresponding elements of the tuple to be comparable; that is, they need to be of the same type. So if the tuples **t1** and **t2** are defined as:

```
t1 = (1, 2, 3, "4", "5")
t2 = (-1,2,4,5,7)
```

then the expression **t1>t2** is not allowed. A string can't be compared against an integer, and the element indexed by 3 of **t1** is a string, whereas the element indexed by 3 of **t2** is an int.

3.4 Lists

One way to think of a Python *list* is that it is a tuple in which the components can be modified. They have many properties of an *array* of the sort one might find in Java or C, in that they can be used as a place to store things and have random access to them; any element can be read or written. They are often used as one might use an array, but have a greater natural functionality.

Initially a list looks like a tuple, but uses square brackets to delimit it.

```
list1 = [2, 3, 5, 7, 11, 13, 17, 19] # Prime numbers under 20
list2 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]
```

```
list3 = ["hi", "ohio", "salut"]
```

A list can be empty:

```
list4 = []
```

and because they are like tuples and strings, each element in a list has an index, and they begin (as usual) at 0. Lists can be indexed and sliced, as before:

```
list1[2:4] is [5, 7]
```

Concatenation is like that of strings too:

```
list6 = list1 + [23, 31]
```

yields [2, 3, 5, 7, 11, 13, 17, 19, 23, 31]

Negative values index from the end of the string. However, unlike strings and tuples, individual elements can be modified. So:

```
list1[2] = 6
```

results in **list1** being [2, 3, 6, 7, 11, 13, 17, 19]. Also:

```
list3[1:] = "bonjour"
```

results in **list3** taking the value—*oops*, it becomes:

```
['hi', "b", "o", "n", "j", "o", "u", "r"].
```

That's because a string is a sequence too, and this string consists of seven components. Each component of the string becomes a component of the list. If the string "bonjour" is supposed to become a single component of the list, then it needs to be done this way:

```
list3[1:] = ["bonjour"]
```

The other components of **list3** are sequences, and now so is the new one. However, integers are *not* sequences, and the assignment:

```
list1[2] = [6,8,9]
```

results in the value of **list2** being:

```
[2, 3, [6, 8, 9], 7, 11, 13, 17, 19]
```

There is a list within this list; that is, the third component of **list1** is not an integer, but is a list of integers. That's legitimate, and works for tuples as well, but may not be what is intended.

Problem: Compute the Average (Mean) of a List of Numbers

The mean is the sum of all numbers in a collection divided by the number of numbers. If a set of numbers already exists as a list, calculating the mean might involve a loop that sums them followed by a division. For example, assuming that **list1** = [2, 3, 5, 7, 11, 13, 17, 19]:

```
mean = 0.0
```

```
for i in list1:
```

```
    mean = mean + i
```

```
mean = mean/len(list1)
```

It can be seen that a list can be used in a loop to define the values that the loop variable `i` will take on, a similar situation to that of a tuple. A second way to do the same thing would be:

```
mean = 0.0
for i in range(0,len(list1)):
    mean = mean + list1[i]
mean = mean/len(list1)
```

In this case the loop variable `i` is an index into the list and not a list element, but the result is the same. Python lists are more powerful than this, and making use of the extensive power of the list simplifies the calculation greatly:

```
mean = sum(list1) / len(list1)
```

The built-in function `sum` will calculate and return the sum of all of the elements in the list. That was the purpose of the loop, so the loop is not needed at all. The functions that work for tuples also work for lists (`min`, `max`, `len`), but some of the power of lists is in the methods it provides.

3.4.1 Editing Lists

Editing a list means to change the values within it, usually to reflect a new situation to be handled by the program. The most obvious way to edit a list is to simply assign a new value to one of the components. For example:

```
list2 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]
list2[0] = "Nitrogen"
print (list2)
```

results in the following output:

```
['Nitrogen', 'Helium', 'Lithium', 'Beryllium', 'Boron', 'Carbon']
```

This substitution of a component is not possible with strings or tuples. It is possible to replace a single component with another list:

```
list2 = ["Hydrogen","Helium","Lithium","Beryllium","Boron"
        "Carbon"]
list2[0] = ["Hydrogen", "Nitrogen"]
```

results in:

```
list2 = [['Hydrogen','Nitrogen'],'Helium','Lithium',
        'Beryllium', 'Boron','Carbon']
```

3.4.2 Insert

This is not normally what is thought of as an insertion, though. To place new components within a list, the **insert** method is provided. This method places a component at a specified index; that is, the index of the new element will be the one given. To place "Nitrogen" at the beginning of list2, which is index 0:

```
list2.insert(0, "Nitrogen")
```

The first value given to **insert**, 0 in this case, is the index at which to place the component, and the second value is the thing to be inserted. Inserting "Nitrogen" at the end of the list would be accomplished by:

```
list2.insert(len(list2), "Nitrogen")
```

However, consider this:

```
list2.insert(-1, "Nitrogen")
```

Will this insert "Nitrogen" at the end? No. At the beginning of the statement, the value of **list2[-1]** is "Carbon." This is the value at index 5. Therefore, the insert of "Nitrogen" will be at index 5, resulting in:

```
['Hydrogen', 'Helium', 'Lithium', 'Beryllium', 'Boron',  
  
'Nitrogen', 'Carbon']
```

3.4.3 Append

Another way to add something to the end of a list is to use the **append** method:

```
list2.append("Nitrogen")
```

Results in:

```
['Hydrogen', 'Helium', 'Lithium', 'Beryllium', 'Boron',  
  
'Carbon', 'Nitrogen']
```

Remember, the "+" operation will only concatenate a list to a list, so the equivalent expression involving "+" would be:

```
list2 = list2 + ["Nitrogen"]
```

3.4.4 Extend

The **extend** method does pretty much the same things as the "+" operator. With the definitions:

```
a = [1,2,3,4,5]
```

```
b = [6,7,8,9,10]
```

```
print (a+b)
```

```
a.extend(b)
```

```
print(a)
```

the output is:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

However, if **append** has been used instead of **extend** above:

```
a = [1,2,3,4,5]
```

```
b = [6,7,8,9,10]
```

```
print (a+b)
```

```
a.append(b)
```

```
print(a)
```

the result would have been:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
[1, 2, 3, 4, 5, [6, 7, 8, 9, 10]]
```

3.4.5 Remove

The remove method does what is expected: it removes an element from the list. But unlike insert, for example, it does not do it using an index; the value to be removed is specified. So:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]

list1.remove("Helium")
```

results in the **list1** being ['Hydrogen', 'Lithium', 'Beryllium', 'Boron', 'Carbon']. Unfortunately, if the component being deleted is not a member of the list, then an error occurs. There are ways to deal with that, or a test can be made for trying to delete an item:

```
if "Nitrogen" in list1:
    list1.remove("Nitrogen")
```

If there is more than a single instance of the item being removed, then only the first one will be removed.

3.4.6 Index

When discussing *tuples* it was learned that the **index** method looked through the tuple and found the index at which a specified item occurred. The **index** method for lists works in the same way. So:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]

print (list1.index("Boron"))
```

prints "4," because the string "Boron" appears at index 4 in this list (starting from 0, of course). If there is more than one occurrence of "Boron" in the list, then the index of the first one (i.e., smallest index) is returned. If the value is not found in the string, then an error occurs. Again, it might be appropriate to check:

```
if "Boron" in list1:
    print (list1.index("Boron"))
```

3.4.7 Pop

The **pop** method is effectively the reverse or inverse of **append**. It removes the last item (i.e., the one having the largest index) from the list. If the list is empty, then an error occurs. For an example:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]

list1.pop()

print (list1)
```

prints the result:

```
['Hydrogen', 'Helium', 'Lithium', 'Beryllium', 'Boron']
```

To avoid the error that can occur if the list is empty, simply check to see that the length of the list is greater than zero before using **pop**:

```
if len(list1) > 0:
    list1.pop()
```

The method is called **pop** because it represents a way to implement the operation of the same name on a data structure called a *stack*.

3.4.8 Sort

This method places the components of a list into ascending order. Using the **list1** variable that has been used so often:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]

list1.sort()

print(list1)
```

the result is:

```
['Beryllium', 'Boron', 'Carbon', 'Helium', 'Hydrogen',
'Lithium']
```

which is in alphabetic order. The method will sort integers and floating point numbers as well. Strings and numbers cannot be mixed, though, because they can't be compared. So:

```
list2 = ["Hydrogen",1,"Helium",2,"Lithium",3,"Beryllium",4,
        "Boron",5]

list2.sort()
```

results in an error that will be something like:

```
list2.sort()

TypeError: unorderable types: int() < str()
```

The meaning of this error should be clear. Things of type **int** (*integer*) and things of type **str** (*string*) can't be compared against each other and so can't be placed in a sensible order if mixed. For sort to work properly, all of the elements of the list must be of the same type. It is always possible to convert one type of thing into another, and in Python converting an integer to a string is accomplished with the **str()** function; string to integer is converted using **int()**. So **str(3)** would result in "3," and **int("12")** is 12. An error will occur if it is not possible, so **int(12.2)** will fail.

If each element of a list is itself a list, it can still be sorted. Consider the list:

```
z = [["Hydrogen",3],["Hydrogen",2],["Lithium",3],
     ["Beryllium",4],["Boron",5]]
```

When sorted this becomes:

```
[['Beryllium',4],['Boron',5],['Hydrogen',2],['Hydrogen',3],
['Lithium',3]]
```

Each component of this list is compatible with the others, consisting of a string and an integer. Thus they can be compared against each other. Notice that there are two entries for hydrogen: one with a number 2 and one with a number 3. The **sort** method arranges them correctly. A list is sorted by individual elements in sequence order, so the first thing tested would be the string. If those are the same, then the next element is checked. That's an integer, so the component with the smallest integer component will come first.

3.4.9 Reverse

In any sequence the order of the components within it is important. Reversing that order is a logical operation to provide, but may not be used very often. One instance

where it can be important is after a sort. The **sort** method always places components into *ascending* order. If they are supposed to be in *descending* order, then the **reverse** method becomes valuable. As an example consider sorting the list **q**:

```
q = [5, 6, 1, 5, 4, 9, 9, 1, 6, 3]
```

```
q.sort()
```

The value of **q** at this point is

```
[1, 1, 3, 4, 5, 5, 6, 6, 9, 9]
```

To place this list in descending order the reverse method is used:

```
q.reverse()
```

and the result is

```
[9, 9, 6, 6, 5, 5, 4, 3, 1, 1]
```

It is hard to say whether ascending order is needed more often than descending order. Names are often sorted smallest first (ascending), but dates are more likely to require more recent dates before later ones (descending).

3.4.10 Count

This method is used to determine how many times a potential component of a list actually occurs. It *does not return the number of elements in the list*—that job is done by the **len** function. Using the list **q** as an example:

```
q = [5, 6, 1, 5, 4, 9, 9, 1, 6, 3]
```

```
print (1,q.count(1), 2, q.count(2), 3, q.count(3), 99,  
      q.count(99))
```

will result in the output:

```
1 2      2 0      3 1      99 0
```

where the spacing is enhanced for emphasis. This says that there are 2 instances of the number 1 (1,2) in the list, zero instances of 2 (2,0), one instance of the number 3 (3,1) and none of 99 (99,0).

3.4.11 List Comprehension

When creating a list of items, two mechanisms have been discussed. The first is to use constants, as in the list **q** in the previous section. The second appends items to a list, and this could be done within a loop. Making a list of perfect squares could be done like this:

```
t = []
```

```
for i in range(0,10):
```

```
    t = t + [i*i]
```

which creates the list `[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]`. This kind of thing is common enough that a special syntax has been created for it in Python—the *list comprehension*.

The basic idea is simple enough, although some specific cases are complicated. In the situation above involving perfect squares, the elements in the list are some function of the index. When that is true the loop, index, and function can be given within the square brackets as a definition of the list. So the list `tt` could be defined as:

```
tt = [i**2 for i in range(10)]
```

The `for` loop is within the square brackets, indicating that the purpose is to define components of the list. The variable `i` here is the loop variable, and `i**2` is the function that creates the elements from the index. This is a simple example of a list comprehension.

Creating random integer values? No problem:

```
tt = [random.randint(0,100) for i in range(10)]
```

The first six elements in all uppercase?

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium",
         "Boron","Carbon"]

ss = [i.upper() for i in list1]
```

This is a very effective way to create lists, but it does depend on having a known connection between the index and the element.

3.4.12 Lists and Tuples

A tuple can be converted into a list. Lists have a greater functionality than tuples; that is, they provide more operations and greater ability to represent data. On the other hand, they are more complicated and require more computer resources. If something can be represented as a tuple, then it is likely best to do so. A tuple is designed to be a collection of elements that as a whole represent some more complicated object, but that individually are perhaps of different types. This is rather like a C *struct* or Pascal *record*. A list is more often used to hold a set of elements that all have the same type, more like an array. This is a good way to think of the two types when deciding what to use to solve a specific problem.

Python provides tools for conversion. The built-in function `list` takes a tuple and converts it into a list; the function `tuple` does the reverse, taking a list and turning it into a tuple. For example, converting `list1` into a tuple:

```
tuple1 = tuple(list1)

print(tuple1)
```

yields

```
('Hydrogen', 'Helium', 'Lithium', 'Beryllium', 'Boron',
'Carbon')
```

This is seen to be a tuple because of the "(" and ")" delimiters. The reverse:

```
v = list(tuple1)

print(v)
```

prints the text line:

```
['Hydrogen', 'Helium', 'Lithium', 'Beryllium', 'Boron',
'Carbon']
```

and the square brackets indicate this is a list.

3.4.13 Exceptions

Exceptions are the usual way to check for errors of indexing and membership in lists. The error is allowed to occur, but an exception is tested and handled in the case where, for example, an item being deleted is not in the list.

Problem: Delete the Element Helium from a List

Earlier, as an example of the remove method, a program snippet was written to delete the element *Helium* from a list of elements:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
"Carbon"]

if "Helium" in list1:

    list1.remove("Helium")
```

Because list1 may not have *Helium* as one of the components, a check was made before an attempt to delete it. An attempt to delete an element from a list where the element does not appear in that list results in an **AttributeError**. Rather than perform an explicit test, a Python programmer would more likely use an exception here. The error can be caught as follows:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
"Carbon"]

try:

    list1.remove("Helium")

except:

    print ('Can't find Helium')
```

The advantage of this over allowing the error to occur is that the program can continue to execute.

Problem: Delete a Specified Element from a List

Given the same list, read an element from the keyboard and delete that element from the list. The basic code is the same, but now the string is entered and could be anything at all. It's easier to test a program when it can be made to fail on purpose. The name is entered using the **input** function and is used as the parameter to **remove**. Now it is possible to test all of the code in this program without changing it. First, here is the program:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]
s = input("Enter:")
try:
    list1.remove(s)
except:
    print ('Can't find ', s)
print (list1)
```

Properly testing a program means executing all of the statements that comprise it and ensuring that the answer given is correct. So in this case, first delete an element that is a part of the list. Try Lithium. Here is the output:

Enter: Lithium

```
["Hydrogen", "Helium", "Beryllium", "Boron", "Carbon"]
```

This is correct. These are the statements that were executed in this instance:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]
s = input("Enter:")
try:
    list1.remove(s) # This was successful
print (list1)
```

Now try to delete *Oxygen*. Output is:

Enter: Oxygen

Can't find Oxygen

```
['Hydrogen', 'Helium', 'Lithium', 'Beryllium', 'Boron', 'Carbon']
```

This is correct. These statements were executed:

```
list1 = ["Hydrogen","Helium","Lithium","Beryllium","Boron",
        "Carbon"]
s = input("Enter:")
try:
    list1.remove(s) # this was not successful
```

```
except:
```

```
    print ('Can't find ', s)
```

```
print (list1)
```

All of the code in the program has been executed and the results checked for both major situations. For any major piece of software this kind of testing is exhausting, but it is really the only way to minimize the errors that remain in the final program.

3.5 SET TYPES

Something of type *set* is an unordered collection of objects. An element can only be a member of a given *set* once, so in that sense it is much like a mathematical set. In fact, that's the point. Because a set is unordered, operations such as indexing and slicing are not provided. It does support membership (**is**), size (**len()**), and looping on membership (**for i in set**) .

Anyone (probably an older person) who knows the *Pascal* language has some familiarity with the set type in Python.

Mathematical sets have certain specific, well-defined operations, and those are available on a Python set also.

Subset **set1 < set2** means **set1** is a true subset of **s2**.

Intersection **set1 & set2** creates a new set containing members in common with both.

Union **set1 | set2** creates a new set with all elements of both.

Difference **set1 - set2** creates a new set with members that are not in both.

Equality **set1 == set2** is true if both sets contain only the same elements.

Creating a new object of type *set* is a matter of specifying either that it is a *set* or what the elements are. So, one way is to use the {} syntax:

```
set1 = {1,3,5,7,9}
```

or to use the constructor:

```
set2 = set(range(1, 10))
```

which gives the set {1, 2, 3, 4, 5, 6, 7, 8, 9}. So:

```
set1 < set2 is True
```

```
set1 & set2 is {9, 1, 3, 5, 7} Note: order does not matter to a set.
```

```
set1 | set2 is {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
set2 - set1 is {8, 2, 4, 6}
```

A new element can be added to a set using **add()**:

```
set1.add(11)
```

and removed using **remove()**:

```
set1.remove(11)
```

or `discard()`:

```
set1.discard(11)
```

If the element being removed is not in the set, then an error will occur (*KeyError*) when `remove()` is called, but not with `discard()`. This should be tested first or be placed in an `except` statement.

All of the examples so far involve integers belonging to a set, but other types can belong as well: floating point numbers, strings, and even tuples (not *lists*). For example, the following are legal sets:

```
{ "a", "e", "i", "o", "u" }

{ "cyan", "yellow", "magenta" }

{ (2,4), (3,9), (4,16), (5,25), (6,36), (7,49) }
```

3.5.1 Example: Craps

Craps is a dice game, for those unfamiliar with it, and commonly involves betting on the outcome. The player (*shooter*) rolls two dice. If, on the first roll (*pass*), a total of 7 or 11 is obtained, then the shooter wins. On the other hand, an initial roll of 2, 3, or 12 loses immediately. Any other roll is called the *point*. In that case the shooter continues to roll the dice. If a 7 is obtained then the shooter loses, and if the point number is rolled then the shooter wins. The shooter continues to roll until one or the other occurs. One way to implement this game in Python is to use sets.

Elements of the sets will be values on each die, which is to say one roll. There are two dice so a total of 36 combinations exist. A single roll is a tuple, such as (1,1) or (3,4). There are only 12 distinct sums of two dice, and multiple ways to achieve them. A sequence named `roll` will be created that contains a set for each possible value, and that set contains all of the ways that the value can be obtained. For instance, there are two ways to roll a 3, so:

```
roll[3] = {(1,2), (2,1)}
```

Initially a set is created for each possible roll of a pair of dice and then is initialized as described:

```
from random import *
```

```
roll = list(range(0,13)) # Create the empty list
for i in range(1,13): # and fill with empty sets.
    roll[i] = set()
for i in range (1,7): # Now for each possible roll
    for j in range (1,7): # of two dice, add that roll
```

```

k = i+j    # to the element of roll for
roll[k].add( (i,j) ) # that value (sum of the
                    # dice)

```

Now `roll[i]` contains all of the ways to roll a value of `i`, In particular, `roll[7]` contains all ways to roll a 7 and `roll[11]` contains all ways to roll an 11. Thus, all of the rolls that will win on the first pass can be placed in a single set, the union of `roll[7]` and `roll[11]`:

```
winner = roll[7] | roll[11]
```

Similarly, the rolls that will lose for the shooter on the first pass are:

```
loser = roll[2] | roll[3] | roll[12]
```

If any other roll is thrown, then that becomes the point. Rolling a die amounts to getting a random number between 1 and 6 inclusive, or:

```

die1 = randrange(1,7)
die2 = randrange(1,7)

```

Remember that `randrange()` produces a number *less* than the second parameter. Given this roll, the point is the set `roll[die1+die2]`. Continuing the program from the die rolls:

```

val = (die1,die2)          # A tuple, the current roll
print ("Shooter rolls ", val)
if val in winner:         # Is this tuple a winner?
    print ("The shooter wins!")
elif val in loser:        # Is it a loser?
    print ("The shooter loses")
else:
    point = roll[die1+die2] # Define the point set
    print (die1+die2, " is your point.")

```

Now the dice are rolled repeatedly. If the roll is in the point set, then the shooter wins. If the roll is a 7 (in the set `roll[7]`) then the player loses. Otherwise the shooter rolls again.

```

while True:              # Repeat until a win or
    # loss happens

    die1 = randrange(1,7) # Roll the dice
    die2 = randrange(1,7)

    val = (die1, die2) # val is a tuple
    print ("Rolls ", val)

```

```

if val in roll[7]: # Any 7 roll loses

    print ("The shooter loses!")

    break

if val in point: # Rolling the 'point' wins.

    print ("The shooter makes the point. A winner!")

    break

```

And that's the game. In a real craps game this entire process is repeated, and bets are placed on each individual game as to whether the player will win or lose.

3.6 SUMMARY

There is a kind type that a variable can have that amounts to a list or sequence of other, simpler, things. This is true, and using variables having these types is an essential part of writing useful and effective code. Python offers *strings*, *tuples*, and *lists* as objects that consist of multiple parts. They are called *sequence* types.

A *string* is a sequence of characters. The word *sequence* implies that the order of the characters within the string matters, and that is true of a string. Strings most often represent the way that communication between a computer and a human takes place. Strings can be indexed to see what character is in any position (e.g., `s[i]`), can be searched for a string that occurs with it, can have characters concatenated to it, and many other useful operations. If a string `s` contains an integer, then `int(s)` will yield that integer and `str(i)` will create a string from an integer `i`.

A *tuple* is almost identical to a string in basic structure, except that it is composed of arbitrary components instead of characters. Examples are `tup1 = (2, 3, 5)` and `tup2 = ("Hydrogen", "Helium", "Carbon")`. A tuple can contain mixed type, such as integers and strings: `tup3 = ("star", 1, "planet", 2)`. An element of a tuple cannot be altered, so it is said to be *immutable*, although concatenation is possible.

A *list* is like a tuple but is not immutable, so individual elements can be modified. A list uses square brackets as a delimiter, instead of parentheses as used for a tuple. Changing an element involves indexing it, so if `list1` is a list then `list1[2] = 6` modifies element 2 of that list.

A *set* is an unordered collection of objects. An element can be almost any type, but can only occur in a set once. This mimics a mathematical set. Elements can be added and removed, and the set operations *union*, *intersection*, and *difference* can be performed.

Exercises

For the exercises below, assume the following definitions:

```
str1 = "okra is the closest thing to nylon i've ever eaten."
```

```
str2 = "pull the string, and it will follow wherever you  
wish."
```

```
str3 = "let out a little more string on your kite."
```

```
str4 = "every string is a different color, a different voice."
```

```
vowels = 'aeiou'
```

```
atoms=("Hydrogen",1,"Helium",2,"Lithium",3,"Boron",5,"Carbon",6, "Oxygen",8)
```

1. What is printed by the following code snippets:

```
a) for i in range(0,len(str3)):
    print (str3[i], end='')
```

```
b) for i in range(0,len(str3)):
    print (i, end='')
```

```
c) for i in range(0,len(str3)):
    print (str2[i], end='')
```

```
d) for i in str3:
    print (i, end='')
```

```
e) for i in str3:
    if i in vowels:
        print(i, end='')
```

```
f) for i in str1:
    if not(i in vowels):
        print(i, end='')
```

2. Construct a loop that prints out all characters of **str4** that correspond to a vowel in **str3**. Note: the two strings are different lengths.

3. A *Caesar cypher* is a way to transmit a secret message. When encoding a message each character is replaced by one that is a fixed distance further along the alphabet. If that distance is 6, for example, the letter "a" would be replaced by "g," which is 6 positions further along. The characters at the end will wrap around to the beginning, so "z" will become "f." Write some Python code that will encode **str1** in this way. Ensure that it works by decrypting the following string:

```
"varr znk yzxotm, gtj oz corr lurruc cnkxkbkx eua coyn."
```

Ans: uqyg oy znk iruykyz znott zu terut o'bk kbkx kgzkt.

4. Write a Python snippet that will create two tuples from the single tuple **atoms**: one named **elements** that contains only the names, and one called **numbers** that contains the atomic numbers of the elements in the tuple **atoms**.
5. Write a Python program that reads numbers from the keyboard and appends them to a tuple. Stop the process when a negative number is entered and then print the tuple that was created.
6. A deck of playing cards consists of 52 items: each one has one of four suits (clubs, diamonds, hearts, and spades) and within each suit are values from 1-10 and "Jack," "Queen," and "King." Write a Python program that creates a deck of cards, shuffles them, and prints out the result.
7. Write a Python program that reads names (single words) one at a time from a keyboard and deletes them from a list named **names** if they are already elements of that list. If the name is not already a member of the list, then it will be added. Typing the name "quit" terminates the program.
8. Assume that a string named **temp** exists and has a value. Write Python code that will print **temp** backwards.
9. A palindrome is a phrase (a string) that reads the same forwards and backwards. The name "hannah" is a palindrome; so is "Ogopogo," the name of a monster who lives in lake Okanogan, and the word "redivider." Write a Python program that determines whether a given string is a palindrome.
10. Most examples of palindromes contain spaces and punctuation, and these characters are ignored when deciding whether or not the phrase is palindromic. So is case. Thus, the phrase "I prefer pi" is a palindrome. With these considerations in mind, write a Python program that determines whether a string is a palindrome or not.

Notes and Other Resources

Built-in types: <https://docs.python.org/3.4/library/stdtypes.html?highlight=set#set>

Python strings: <https://docs.python.org/3/library/string.html>

Rules of Craps: <http://www.bigmcasino.com/learn-more/learn-to-play-craps/what-are-the-basic-rules-of-craps/>

1. David Mertz. (2003). **Text Processing in Python**, Addison Wesley Professional, ISBN-13: 978-0321112545.
2. David Makinson. (2012). **Sets, Logic, and Maths for Computing**, 2nd edition, Springer, ISBN-13: 978-1447124993.
3. J. D. Oldham. (2005). **What happens after Python in CS1?** *Journal of Computing Sciences in Colleges*, 20 (6), 7-13.